

5.4 Documento de conclusiones (Post-Mortem)

5.4.1 Comparativa de tiempos y costes

El diagrama de Gantt es prácticamente idéntico al que se planificó desde el principio ya que no ha surgido ningún retraso, Las únicas dos desviaciones son menores: **1.1.1** arrancó el día 3 en lugar del día 1 (sin impacto, ya que corría en paralelo con A dentro de la misma ventana temporal), y **1.3.1** se ejecutó el 12 de mayo en lugar del 9-11, solapándose con los Scripts SQL de Rubén en lugar de precederlos. Ninguna de las dos afectó al camino crítico ni a la entrega final.

Horas reales por integrante según GrindStone

Integrante	Tareas registradas	Horas reales
Rubén Albarrán Martín	3.1.1, 4.1.1, 5.1.1, 6.1.1, 2.1.2, 2.2.1, 1.3.1, 1.3.2	23 h
Ian García Mateo	1.1.4,3.2.1,4.2.1,5.2.1,6.2.1,3.3.1,4.3.1,5.3.1,6.3.1,6.3.4	22 h
Daniel Maroto López	1.1.1, 1.1.2, 1.1.3, 1.2.2, 1.3.4	21 h
Miguel Pina Gomez	1.1.4, 1.2.1, 2.1.1, 3.2.2, 4.2.2, 3.3.2, 4.3.2, 5.3.2, 6.3.2, 6.3.3, 1.3.3	19 h
Total		85

Líneas de código reales del sistema

Componente	Líneas
Frontend	11.014
Backend	438
Base de datos	111
Total	11.563 líneas = 11,563 KLOC

Presupuesto en Eurípides

El proyecto duró 41 días = 1,37 meses

Concepto	Cálculo	Resultado
Presupuesto total	1000€P * 1,37 meses * 4P	5.480€P
horas reales	23h + 22h + 19h + 21h	85 horas
Valor-hora real estimado	5480€P / 85h	64,47€/h

El modelo COCOMO está diseñado para proyectos industriales de gran escala, no para prácticas académicas ágiles. Esto explica que nuestro valor-hora real se haya disparado a 64,47 €/h

A nivel colectivo, la auditoría de los registros de GrindStone revela un éxito rotundo en la gestión de recursos humanos y asignación de tareas. A diferencia de lo que suele ocurrir en proyectos académicos, donde se producen grandes desequilibrios de carga, este equipo ha funcionado con una equidad sobresaliente. La diferencia entre el integrante con más horas (23 horas) y el de menor carga (19 horas) es de apenas 4 horas en un ciclo de vida de mes y medio. Esto demuestra que la división modular planteada en la WBS de la Fase 1 fue realista y que el compromiso ha sido uniforme en todas las fases del desarrollo.

Desglose y evaluación por trabajador:

- **Rubén Albarrán (23h):** Como *Data Analyst & QA*, Rubén lidera el cómputo global de horas del proyecto. Su carga se justifica por haber asumido responsabilidades críticas en los extremos del proyecto: al inicio, detallando exhaustivamente los Casos de Uso; y al final, asumiendo la ejecución de las Pruebas de Integración y el aseguramiento de la calidad (Testing). Su precisión al redactar el modelo ER y los Scripts SQL (en apenas 111 líneas operativas) evitó bloqueos en el resto del equipo.
- **Ian García (22h):** En su rol de *Frontend & UX/UI*, la dedicación de Ian ha sido constante y de un altísimo rendimiento. Sus 22,5 horas de trabajo registrado soportan el mayor peso volumétrico del proyecto: la producción de más de 11.000 líneas de código cliente. Su esfuerzo no solo abarcó el diseño de los wireframes, sino la compleja implementación de la PWA, la gestión de la interfaz plurilingüe y la adaptación *responsive* de todos los módulos.
- **Daniel Maroto (21h):** Como *Project Manager*, Daniel ha mantenido el control estructural del proyecto rozando las 20 horas de dedicación. Su carga temporal se ha concentrado de manera estratégica en la Fase 1 (Gantt, PERT, Riesgos) y Fase 2 (Estimación COCOMO y Presupuesto), culminando con la redacción de este análisis Post-Mortem. Su eficiencia en la gestión ha garantizado que el equipo técnico tuviera los requisitos claros y trabajara sin bloqueos administrativos.
- **Miguel Pina (19h 00m):** Como *Backend & SCM*, Miguel registra la menor carga horaria formal, pero esconde el mayor nivel de eficiencia técnica del grupo. Lograr implementar la lógica del servidor, el sistema de autenticación, la API REST y el despliegue del contenedor Docker en apenas 19 horas y con solo 438 líneas de código Python, demuestra un dominio absoluto de la arquitectura backend, priorizando la calidad y la optimización del código frente al volumen.

5.4.2 Desviaciones respecto al diseño original

Funcionalidades implementadas vs. requisitos iniciales

El sistema implementado cubre los once requisitos funcionales definidos en la Fase 1. RF-01 (gestión de identidades), RF-02 (dashboard), RF-03 (catálogo), RF-04 (noticias), RF-05 (blog y comentarios), RF-06 (hub multimedia), RF-07 (agenda), RF-09 (formulario de contacto), RF-10 (plurilingüe) y RF-11 (responsive) están presentes y operativos. RF-08 (página del equipo de redacción) fue implementada con un nivel de detalle reducido respecto al diseño original para priorizar los módulos del camino crítico en los últimos días de desarrollo.

Cambios en arquitectura y tecnologías

El stack tecnológico no sufrió modificaciones significativas respecto al planificado en la Fase 1. Python como backend, MySQL como base de datos y Docker para el entorno de servidor funcionaron según lo previsto. La única desviación notable fue en el frontend: la implementación de la PWA mediante Service Workers resultó más compleja de lo estimado, lo que llevó al equipo a simplificar el comportamiento offline, entregando la funcionalidad de instalación en pantalla de inicio sin caché avanzada de contenido.

Tareas de la WBS con desviación temporal

Tarea	Estimación original	Tiempo real	Desviación
1.1.2	6 días	10,5h	Superior a lo estimado
1.2.2	5 días	6 h en dos sesiones	Dentro del margen
2.2.1	2 días	4 h	Dentro del margen
1.3.2	2 días	3,5 h	Dentro del margen

La tarea que más se desvió fue 1.1.2, donde Daniel empleó significativamente más tiempo del planificado debido a las múltiples revisiones del Gantt y el PERT a lo largo de todo el proceso.

5.4.3 Evaluación de riesgos

R01 — Retraso en los Scripts SQL No se materializó. Rubén completó los scripts el 9 y 11 de mayo según lo planificado, desbloqueando a Ian y Miguel a tiempo. La medida preventiva de validar el ER con DB Fiddle durante la Fase 2 fue determinante para que el proceso fuera fluido.

R02 — Incompatibilidad Frontend / Backend Se materializó de forma leve durante la integración del módulo de autenticación. El contrato de API acordado previamente permitió resolver el desajuste en pocas horas sin impacto en el calendario. La sesión conjunta de alineación prevista en el plan de contingencia fue necesaria pero breve.

R03 — Baja disponibilidad de un integrante No se materializó de forma crítica. Hubo días puntuales con menor disponibilidad individual por otras asignaturas, pero nunca coincidieron con tareas del camino crítico. La documentación detallada del WBS y los commits diarios permitieron mantener la continuidad sin pérdidas.

R04 — Desconocimiento de tecnologías clave Se materializó parcialmente con la PWA. Ian encontró dificultades con los Service Workers que llevaron a simplificar el comportamiento offline tal como contemplaba el plan de contingencia. Docker no generó problemas relevantes gracias a la estandarización del docker-compose.yml por parte de Miguel desde el inicio de la Fase 3.

R05 — Pérdida de código en el repositorio No se materializó. La política de ramas GitFlow y los pushes obligatorios al finalizar cada sesión de GrindStone protegieron el código en todo momento.

R06 — Problemas de configuración del entorno Docker Se materializó de forma leve en el equipo de uno de los integrantes (diferencias entre Windows y macOS). Se resolvió en menos de dos horas siguiendo el plan de contingencia: usar el docker-compose.yml de referencia de Miguel como base y ajustar variables de entorno locales.

Riesgo imprevisto — Volumen del frontend No fue identificado en el análisis inicial. El frontend alcanzó las 11.014 líneas de código, muy por encima de lo esperado para un proyecto de esta escala. Aunque no comprometió los plazos gracias a la experiencia de Ian con HTML/CSS/JS, sí generó una asimetría notable entre el volumen de trabajo del frontend y el backend (438 líneas), lo que debería haberse previsto en la estimación KLOC.

5.4.4 Reflexión final del equipo

Problemas de comunicación y coordinación

La coordinación entre Ian (frontend) y Miguel (backend) requirió más atención de la prevista durante la semana del 12 al 19 de mayo. Los Diagramas de Secuencia de la Fase 2 resultaron insuficientes como contrato de API completo, y fue necesario complementarlos con acuerdos informales por Discord durante la implementación. En retrospectiva, habría sido útil definir un documento de endpoints explícito antes de arrancar la Fase 3.

La comunicación general del equipo funcionó correctamente mediante Discord y WhatsApp. Las reuniones de coordinación fueron ágiles y los bloqueos se resolvieron con rapidez.

Lecciones aprendidas

La planificación con WBS y Gantt demostró ser útil para mantener el orden del proyecto, aunque el nivel de detalle del Gantt fue difícil de mantener actualizado en tiempo real. El PERT resultó especialmente valioso para identificar el camino crítico y anticipar que cualquier retraso en los Scripts SQL de Rubén era el mayor riesgo operativo del proyecto, lo que llevó al equipo a priorizarlo correctamente.

GrindStone como herramienta de control de horas aportó una visión objetiva del esfuerzo real por integrante, revelando que la carga fue relativamente equilibrada entre Daniel, Miguel y Rubén (aproximadamente 20 horas cada uno), mientras que el volumen de código de Ian sugiere una carga de trabajo superior no completamente reflejada en el registro hasta la fecha de este análisis.

Propuestas de mejora

Si el proyecto se volviera a realizar, el equipo aplicaría tres mejoras concretas. En primer lugar, definir el contrato de API entre frontend y backend en la Fase 2 como un documento independiente y obligatorio, no solo como diagrama de secuencia. En segundo lugar, estimar KLOC de forma más conservadora para el frontend, que en proyectos web tiende a crecer muy por encima de lo previsto. En tercer lugar, iniciar el registro de GrindStone desde el primer día sin excepción, ya que las primeras sesiones de configuración y aprendizaje de herramientas suelen no registrarse y generan huecos en el historial de horas.